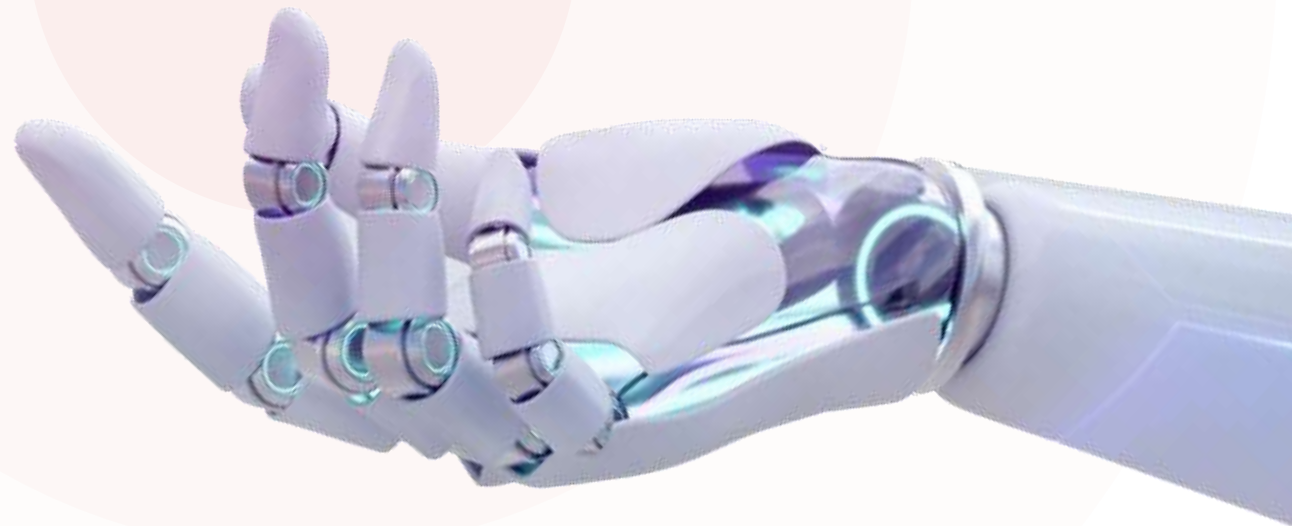




面向对象高级

final、单例类、枚举类、抽象类、接口



多一句没有，少一句不行，用更短的时间，教会更实用的技术

CONTENT

面向对象编程

➤ **final**

认识 final 关键字

常量

➤ 单例类

➤ 枚举类

➤ 抽象类

➤ 接口

➤ 综合小案例





认识 final

final 关键字是最终的意思，可以修饰：类、方法、变量。

- 修饰类：该类被称为最终类，特点是不能被继承了。
- 修饰方法：该方法被称为最终方法，特点是不能被重写了。
- 修饰变量：该变量有且仅能被赋值一次。

final 修饰变量的注意点

- final 修饰基本类型的变量，变量存储的**数据**不能被改变。
- final 修饰引用类型的变量，变量存储的**地址**不能被改变，但地址所指向对象的内容是可以被改变的。

CONTENT

面向对象编程

➤ **final**

认识 final 关键字

常量

➤ 单例类

➤ 枚举类

➤ 抽象类

➤ 接口

➤ 综合小案例





常量

- 使用了 `static final` 修饰的成员变量就被称为常量。
- 作用：常用于记录系统的配置信息。

```
public class Constant {  
    public static final String SCHOOL_NAME = "传智教育";  
}
```

注意！常量名的命名规范：建议使用大写英文单词，多个单词使用下划线连接起来。

使用常量记录系统配置信息的优势、执行原理

- 代码可读性更好，可维护性也更好。
- 程序编译后，常量会被“宏替换”：出现常量的地方全部会被替换成其记住的字面量，这样可以保证使用常量和直接用字面量的性能是一样的。

CONTENT

面向对象编程

- final
- **单例类 (设计模式)**
- 枚举类
- 抽象类
- 接口
- 综合小案例

架构师

面试

源码





什么是设计模式？

- 一个问题通常有 n 种解法，其中肯定有一种解法是最优的，这个最优的解法被人总结出来了，称之为**设计模式**。
- 设计模式有 20 多种，对应 20 多种软件开发中会遇到的问题。

关于设计模式，主要学什么？

01

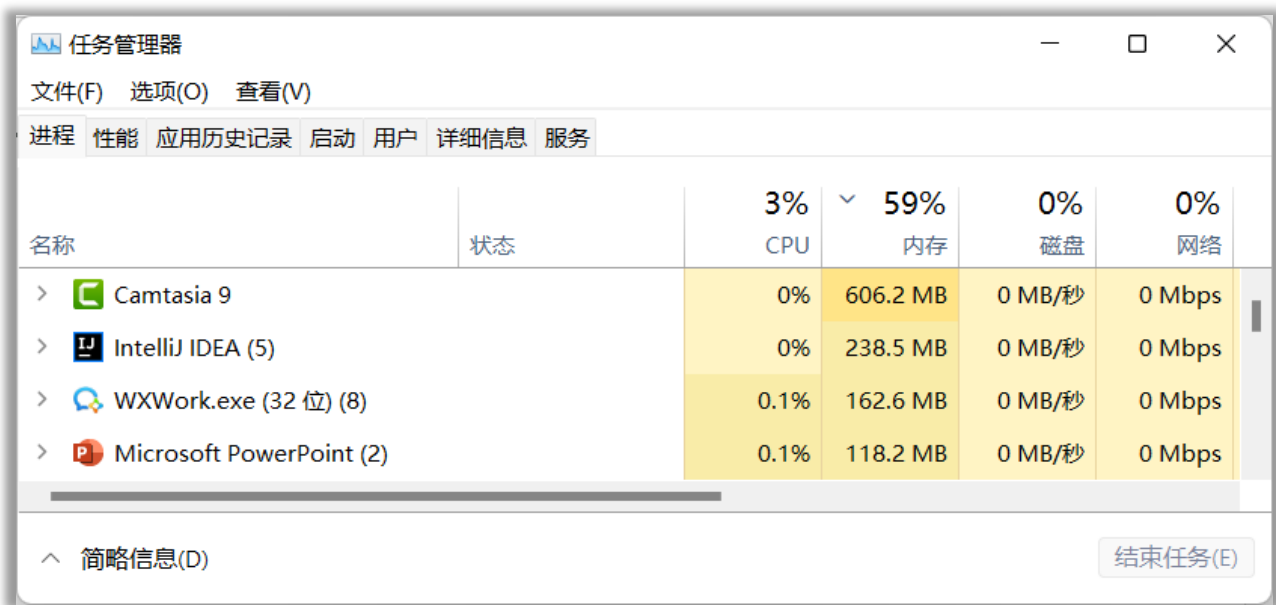
解决什么问题？

02

怎么写？

单例设计模式

- 作用：确保某个类只能创建一个对象。



任务管理器

文件(F) 选项(O) 查看(V)

进程 性能 应用历史记录 启动 用户 详细信息 服务

名称	状态	3% CPU	59% 内存	0% 磁盘	0% 网络
> Camtasia 9		0%	606.2 MB	0 MB/秒	0 Mbps
> IntelliJ IDEA (5)		0%	238.5 MB	0 MB/秒	0 Mbps
> WXWork.exe (32 位) (8)		0.1%	162.6 MB	0 MB/秒	0 Mbps
> Microsoft PowerPoint (2)		0.1%	118.2 MB	0 MB/秒	0 Mbps

^ 简略信息(D) 结束任务(E)

单例设计模式

- 作用：确保某个类只能创建一个对象。

写法，实现步骤：

- 把类的构造器私有。
- 定义一个静态变量记住类的一个对象。
- 定义一个静态方法，返回对象。

```
// 单例类
public class A {
    // 2、定义一个静态变量记住类的一个对象
    private static A a = new A();

    // 1、私有构造器
    private A() {
    }

    // 3、定义一个静态方法返回对象
    public static A getObject() {
        return a;
    }
}
```

单例类有很多形式

- 饿汉式单例：拿对象时，对象早就创建好了。
- 懒汉式单例：拿对象时，才开始创建对象。
- ...
- ...
- ...

```
public class A {  
    // 2、定义一个类变量记住类的一个对象  
    private static A a = new A();  
  
    // 1、私有构造器  
    private A() {  
    }  
  
    // 3、定义一个类方法返回对象  
    public static A getObject() {  
        return a;  
    }  
}
```

什么是设计模式，设计模式主要学什么？ 单例模式解决了什么问题？

- 具体问题的最优解决方案。
- 解决了什么问题？ 怎么写？
- 确保某个类只能创建一个对象。

BRIEF 小结

单例怎么写？ 饿汉式单例的特点是什么？

- 把类的构造器私有；定义一个静态变量存储类的一个对象；提供一个静态方法返回对象。
- 在获取类的对象时，对象已经创建好了。

单例有啥应用场景，有啥好处？

- 任务管理器对象、获取运行时对象。
- 在这些业务场景下，使用单例模式，可以避免浪费内存。



懒汉式单例类

- 用对象时，才开始创建对象。

写法

- 把类的构造器私有。
- 定义一个静态变量用于存储对象。
- 提供一个静态方法，保证返回的是同一个对象。

```
public class B {  
    // 2、定义一个类变量用于存储对象  
    private static B b ; // null  
  
    // 1、单例必须私有构造器  
    private B() {  
    }  
  
    // 3、提供一个类方法返回类的一个对象  
    public static B getObject() {  
        if (b == null) {  
            b = new B();  
        }  
        return b;  
    }  
}
```



懒汉单例模式的特点是什么？

- 要用类的对象时才创建对象（延迟加载对象）

懒汉单例模式怎么写？

- 把构造器私有。
- 定义一个类变量用于存储对象。
- 提供一个类方法，保证返回的是同一个对象。

BRIEF
小结



CONTENT

面向对象编程

➤ final

➤ 单例类

➤ 枚举类

认识枚举类

枚举类的常见应用场景

➤ 抽象类

➤ 接口

➤ 综合小案例





枚举类

- 枚举是一种特殊类。

枚举类的写法

```
修饰符 enum 枚举类名 {  
    名称 1, 名称 2, ...;  
    其他成员...  
}
```

```
public enum A {  
    X , Y , Z;  
    ...  
}
```

特点:

- 枚举类中的第一行，只能写枚举类的对象名称，且要用逗号隔开。
- 这些名称，本质是常量，每个常量都记住了枚举类的一个对象。

枚举类的特点

```
public enum A {  
    X , Y , Z;  
}
```

```
// Compiled from "A.java"  
public final class A extends java.lang.Enum<A> {  
    public static final A X = new A();  
    public static final A Y = new A();  
    public static final A Z = new A();  
  
    public static A[] values();  
    public static A valueOf(java.lang.String);  
}
```

- 枚举都是最终类，不可以被继承，枚举类都是继承 java.lang.Enum 类的。
- 枚举类的第一行只能罗列一些名称，这些名称都是常量，并且每个常量会记住枚举类的一个对象。
- 枚举类的构造器都是私有的（写不写都只能是私有的），因此，枚举类对外不能创建对象。
- 编译器为枚举类新增了几个方法。

CONTENT

面向对象编程

➤ final

➤ 单例类

➤ 枚举类

认识枚举类

枚举类的常见应用场景

➤ 抽象类

➤ 接口

➤ 综合小案例



枚举类的常见应用场景

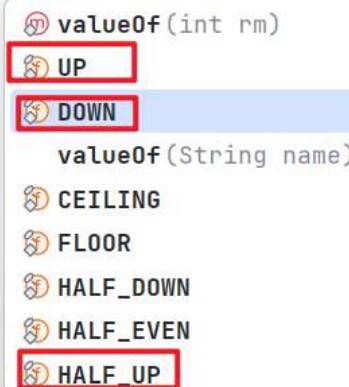
- 枚举类很适合做信息分类和标志。



```
public enum Direction {  
    UP, DOWN, LEFT, RIGHT  
}
```

```
public void move(方向) {  
    根据用户要求的方向控制图块移动  
}
```

```
BigDecimal a = BigDecimal.valueOf(0.1);  
BigDecimal b = BigDecimal.valueOf(0.3);  
BigDecimal result = a.divide(b, scale: 2, RoundingMode.);  
System.out.println(result);
```



CONTENT

面向对象编程

➤ final

➤ 单例类

➤ 枚举类

➤ 抽象类

◆ 认识抽象类

◆ 使用抽象类的好处

◆ 模板方法设计模式

➤ 接口

➤ 综合小案例





抽象类

- 在 Java 中有一个关键字叫: **abstract**, 它就是抽象的意思, 可以用它**修饰类**、**成员方法**。
- **abstract** 修饰类, 这个类就是**抽象类**。
- **abstract** 修饰方法, 这个方法就是**抽象方法**。

```
修饰符 abstract class 类名 {  
    修饰符 abstract 返回值类型 方法名称(形参列表);  
}
```

```
public abstract class A {  
    // 抽象方法: 必须abstract修饰, 只有方法签名, 不能有方法体  
    public abstract void test();  
}
```



抽象类的注意事项、特点

- 抽象类中不一定要有抽象方法，有抽象方法的类必须是抽象类。
- 类有的成员：成员变量、方法、构造器，抽象类都可以有。
- **抽象类最主要的特点：**抽象类不能创建对象，仅作为一种特殊的父类，让子类继承并实现。
- 一个类继承抽象类，必须重写完抽象类的全部抽象方法，否则这个类也必须定义成抽象类。



BRIEF 小结

1、抽象类、抽象方法是什么样的？

- 都是用 `abstract` 修饰的；抽象方法只有方法签名，不能写方法体。

2、抽象类有哪些注意事项和特点？

- 抽象类中可以不写抽象方法，但有抽象方法的类必须是抽象类
- 类有的成员（成员变量、方法、构造器）抽象类都具备。
- 抽象类不能创建对象，仅作为一种特殊的父类，让子类继承并实现。
- 一个类继承抽象类，必须重写完抽象类的全部抽象方法，否则这个类也必须定义成抽象类。



CONTENT

面向对象编程

- final
- 单例类
- 枚举类
- **抽象类**
 - ◆ 认识抽象类
 - ◆ 使用抽象类的好处
 - ◆ 模板方法设计模式
- 接口
- 综合小案例



使用抽象类的好处

- 父类知道每个子类都要做某个行为，但每个子类要做的情況不一样，父类就定义成抽象方法，交给子类去重写实现，我们设计这样的抽象类，就是为了更好的支持多态。

需求

某宠物游戏，需要管理猫、狗的数据。

猫的数据有：名字；行为是：喵喵喵的叫~

狗的数据有：名字；行为是：汪汪汪的叫~

请用面向对象编程设计该程序。

爸爸!

```
public abstract class Animal {  
    private String name;  
  
    public abstract void cry();  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

爸爸!

```
public class Dog extends Animal {  
  
    @Override  
    public void cry() {  
        System.out.println(getName()+ "汪  
        汪汪的叫~~~");  
    }  
}
```

```
public class Cat extends Animal {  
  
    @Override  
    public void cry() {  
        System.out.println(getName()+ "喵  
        喵喵的叫~~~");  
    }  
}
```

BRIEF 小结

1、抽象类的应用场景和好处是什么？

- 父类知道每个子类都要做某个行为，但每个子类要做情况不一样，父类就定义成抽象方法，交给子类去重写实现，**我们抽出这样的抽象类，就是为了更好的支持多态。**

CONTENT

面向对象编程

➤ final

➤ 单例类

➤ 枚举类

➤ **抽象类**

◆ 认识抽象类

◆ 使用抽象类的好处

◆ 模板方法设计模式

➤ 接口

➤ 综合小案例

模板方法设计模式

- 提供一个方法作为完成某类功能的模板，模板方法封装了每个实现步骤，但允许子类提供特定步骤的实现。
- 模板方法设计模式可以：提高代码的复用、并简化子类设计。

```
public class A {  
    ...  
  
    public void write() {  
        代码一样  
  
        代码不同  
  
        代码一样  
    }  
    ...  
}
```

```
public class B {  
    ...  
  
    public void write() {  
        代码一样  
  
        代码不同  
  
        代码一样  
    }  
    ...  
}
```



模板方法设计模式

- 提供一个方法作为完成某类功能的模板，模板方法封装了每个实现步骤，但允许子类提供特定步骤的实现。
- 模板方法设计模式可以：提高代码的复用、并简化子类设计。

写法

- 1、定义一个抽象类。
- 2、在里面定义 2 个方法
 - 一个是模板方法：把共同的实现步骤放里面去。
 - 一个是抽象方法：不确定的实现步骤，交给具体的子类来完成。



建议使用 final 关键字修饰模板方法，为什么？

- 模板方法是给子类直接使用的，不能被子类重写。
- 一旦子类重写了模板方法，模板方法就失效了。



BRIEF 小结

1、模板方法设计模式解决了什么问题？

- 解决方法中存在重复代码的问题。

2、模板方法设计模式应该怎么写？

- 定义一个抽象类。
- 在里面定义 2 个方法，一个是模板方法：放相同的代码里，一个是抽象方法：具体实现交给子类完成。

3、模板方法建议使用什么关键字修饰？为什么

- 建议使用 final 关键字修饰模板方法。

CONTENT

面向对象编程

➤ final

➤ 单例类

➤ 枚举类

➤ 抽象类

➤ **接口**

➤ 综合小案例

◆ 接口概述

◆ 接口好处

◆ 接口的综合小案例

◆ JDK 8 开始，接口新增的三种方法

◆ 接口的注意事项

接口的概述

- Java 提供了一个关键字 `interface` 定义出接口。

```
public interface 接口名 {  
    // 成员变量（常量）  
    // 成员方法（抽象方法）  
}
```

- 注意：接口不能创建对象。
- 接口是用来被类**实现**（implements）的，实现接口的类称为**实现类**，一个类可以同时实现多个接口。

```
修饰符 class 实现类类名 implements 接口1, 接口2, 接口3, ... {  
    // 实现类实现多个接口，必须重写完全部接口的全部抽象方法，否则实现类需要定义成抽象类。  
}
```

1、接口是什么？

- 使用 `interface` 关键字定义的一种结构，JDK 8 之前，接口中只能定义成员变量和成员方法。

小结

2、接口怎么使用？需要注意什么？

- 接口 是被类 实现的。

```
修饰符 class 实现类类名 implements 接口1, 接口2, 接口3 , ... {  
    //实现类实现多个接口，必须重写完全部接口的全部抽象方法，否则实现类需要定义成抽象类。  
}
```

CONTENT

面向对象编程

➤ final

➤ 单例类

➤ 枚举类

➤ 抽象类

➤ **接口**

➤ 综合小案例

◆ 接口概述

◆ **接口好处**

◆ 接口的综合小案例

◆ JDK 8 开始，接口新增的三种方法

◆ 接口的注意事项



接口的好处

- 弥补了类单继承的不足，一个类同时可以实现多个接口，使类的角色更多，功能更强大。
- 让程序可以面向接口编程，这样程序员就可以灵活方便的切换各种业务实现（更利于程序的解耦合）。

CONTENT

面向对象编程

- final
- 单例类
- 枚举类
- 抽象类
- **接口**
 - ◆ 接口概述
 - ◆ 接口好处
 - ◆ **接口的综合小案例**
 - ◆ JDK 8 开始，接口新增的三种方法
 - ◆ 接口的注意事项
- 综合小案例

需求

请设计一个班级学生的信息模块：学生的数据有：姓名、性别、成绩

功能 1：要求打印出全班学生的信息； 功能2：要求打印出全班学生的平均成绩。

注意！ 以上功能的业务实现是有多套方案的，比如：

第 1 套方案：能打印出班级全部学生的信息；能打印班级全部学生的平均分。

第 2 套方案：能打印出班级全部学生的信息（**包含男女人数**）；能打印班级全部学生的平均分（**要求是去掉最高分、最低分**）。

要求：系统可以支持灵活的切换这些实现方案。

CONTENT

面向对象编程

➤ final

➤ 单例类

➤ 枚举类

➤ 抽象类

➤ **接口**

➤ 综合小案例

◆ 接口概述

◆ 接口好处

◆ 接口的综合小案例

◆ JDK 8 开始，接口新增的三种方法

◆ 接口的注意事项

接口新增的三种方法

JDK 8 开始，接口新增了三种形式的方法：

```
public interface A {  
    /**  
     * 1、默认方法（实例方法）：使用default修饰，默认会被加上public修饰。  
     * 注意：只能使用接口的实现类对象调用  
     */  
    default void test1() {  
        ...  
    }  
  
    /**  
     * 2、私有方法：必须用private修饰（JDK 9开始才支持）  
     */  
    private void test2() {  
        ...  
    }  
  
    /**  
     * 3、类方法（静态方法）：使用static修饰，默认会被加上public修饰。  
     * 注意：只能用接口名来调用。  
     */  
    static void test3() {  
        ...  
    }  
}
```

JDK 8 开始，接口中为啥要新增这些方法？

- 增强了接口的能力，更便于项目的扩展和维护。

BRIEF 小结

1、JDK 8 开始，接口中新增了哪些方法？

- 默认方法：使用 default 修饰，使用实现类的对象调用。
- 静态方法：static 修饰，必须用当前接口名调用
- 私有方法：private 修饰，JDK 9 开始才有的，只能在接口内部被调用。
- 他们都会默认被 public 修饰。

2、JDK 8 开始，接口中为啥要新增这些方法？

- 增强了接口的能力，更便于项目的扩展和维护。

CONTENT

面向对象编程

- final
- 单例类
- 枚举类
- 抽象类
- **接口**
 - ◆ 接口概述
 - ◆ 接口好处
 - ◆ 接口的综合小案例
 - ◆ JDK 8开始，接口新增的三种方法
 - ◆ **接口的注意事项**
- 综合小案例



接口的注意事项

- 1、接口与接口可以多继承：一个接口可以同时继承多个接口【重点】。
- 2、一个接口继承多个接口，如果多个接口中存在方法签名冲突，则此时不支持多继承，也不支持多实现。
- 4、一个类继承了父类，又同时实现了接口，如果父类中和接口中有同名的默认方法，实现类会优先用父类的。
- 5、一个类实现了多个接口，如果多个接口中存在同名的默认方法，可以不冲突，这个类重写该方法即可。

CONTENT

面向对象编程

- final
- 单例类
- 枚举类
- 抽象类
- 接口
- 综合小案例





- 某智能家居系统，可以让用户选择要控制的家用设备（吊灯，电视机，洗衣机，落地窗），并可以对它们进行打开或者关闭操作。